

Query/Answer Strategies Comparison

"The search for the perfect language is the story of a dream and of a series of failures."

— Umberto Eco, *The Search for the Perfect Language*

Historical note: Leibniz's *characteristica universalis* assigned prime numbers to primitive concepts — "animal" = 2, "rational" = 3 — so that composite concepts became products: "human" = $2 \times 3 = 6$. Reasoning reduced to divisibility tests. Modern embeddings pursue the same dream: mapping meaning to vectors where semantic relationships become geometric operations (cosine similarity). The difference is that Leibniz required *a priori* enumeration of all primitive ideas, while embeddings learn structure statistically from data.

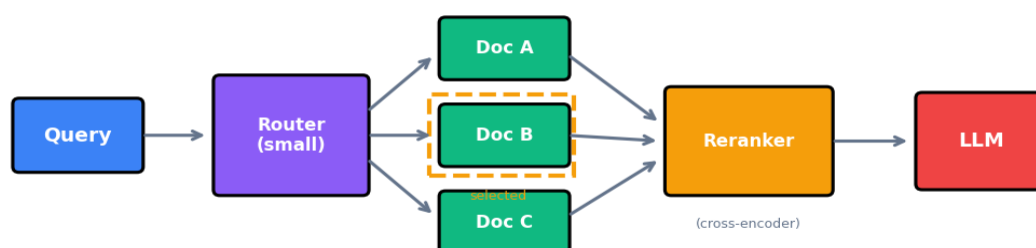
OpenRouter API Token:

```
k-or-v1-c2b3fed13934605c6b75e9962fbef4b867bf2da694d1661f5d88229cda0da3c
```

Acronyms & Terms:

- **RAG** — Retrieval-Augmented Generation: combining retrieval with LLM generation
- **BM25** — Best Matching 25: classic keyword-based ranking algorithm (TF-IDF variant)
- **RAPTOR** — Recursive Abstractive Processing for Tree-Organized Retrieval
- **ColPali** — Vision-language model for visual document embeddings (based on PaliGemma)
- **LLM** — Large Language Model
- **VLM** — Vision-Language Model
- **Cross-encoder** — Reranker model that scores query-document pairs jointly

Strategy 1: Small Router Model

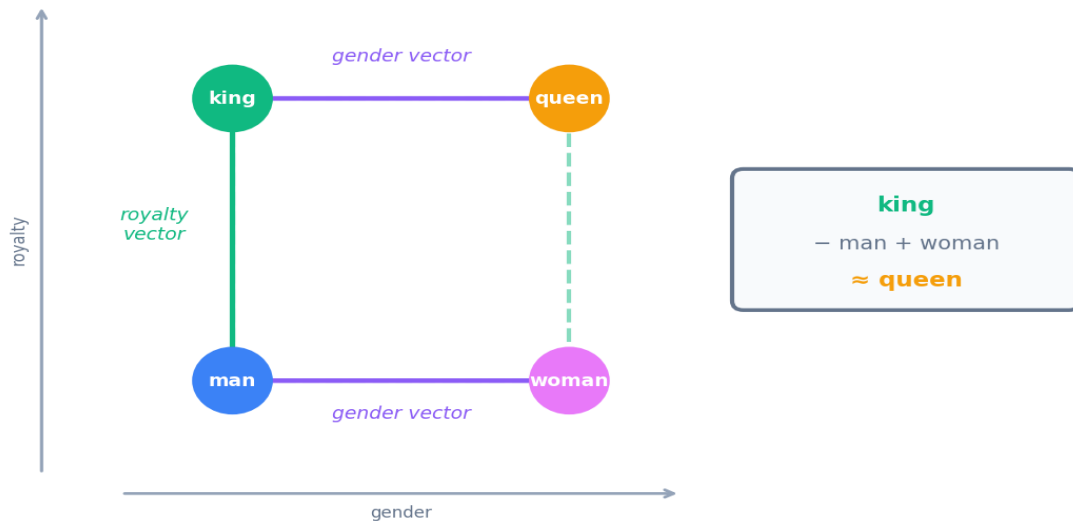


Routes query to most relevant document(s) using a lightweight classifier, then reranks before LLM.

Understanding Word Vectors

Word embeddings map words to vectors in high-dimensional space, where **semantic relationships become geometric relationships**. The classic example: the vector from "man" to "woman" captures the concept of gender — and this same vector connects "king" to "queen".

Word Vectors: Semantic Relationships as Geometry



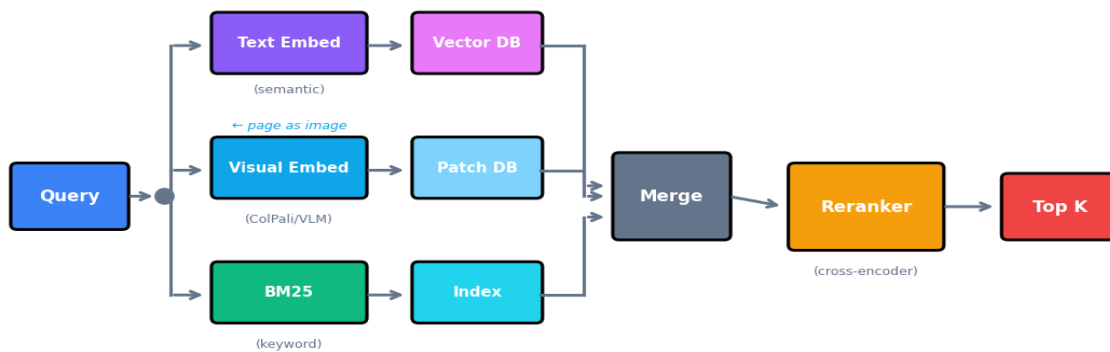
The famous equation: $\text{king} - \text{man} + \text{woman} \approx \text{queen}$

This works because embeddings encode *relational structure*. The "royalty" dimension is preserved while the "gender" dimension shifts. This is why vector similarity (cosine distance) can capture meaning — semantically related words cluster together, and analogies become vector arithmetic.

From words to documents: RAG systems extend this idea. Instead of single words, we embed chunks or entire pages, then find the most similar vectors to a query. The same geometric intuition applies — relevant documents are "nearby" in embedding space.

Strategy 2: RAG + BM25 + Visual Embeddings

Strategy 2: RAG + BM25 + Visual Embeddings

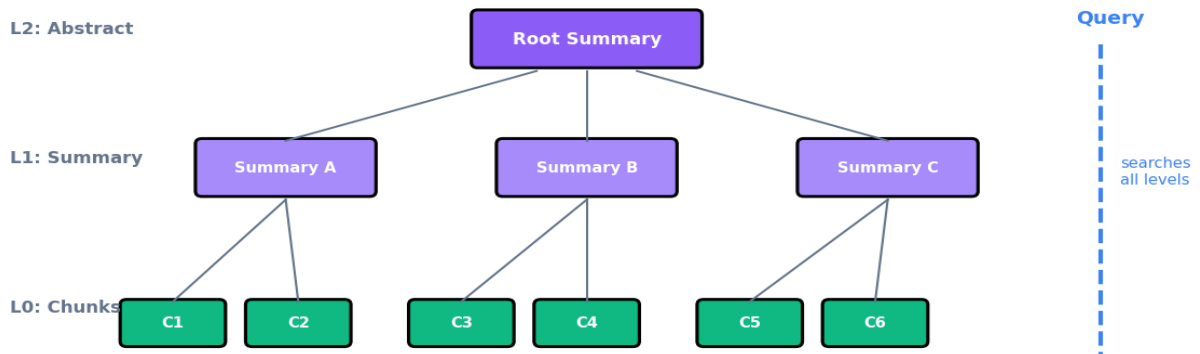


Three parallel paths: text embeddings (semantic), visual embeddings (ColPali — page as image), and BM25 (keyword). Results merged and reranked.

Strategy 3: RAPTOR

"The defect of all symbolic systems lies in their inability to account for context." — Umberto Eco

Strategy 3: RAPTOR (Hierarchical Summarization)



Builds a hierarchical tree of summaries. Query searches all levels — chunks, summaries, and abstract root.

Strategy 5: Recursive Language Models (RLM)

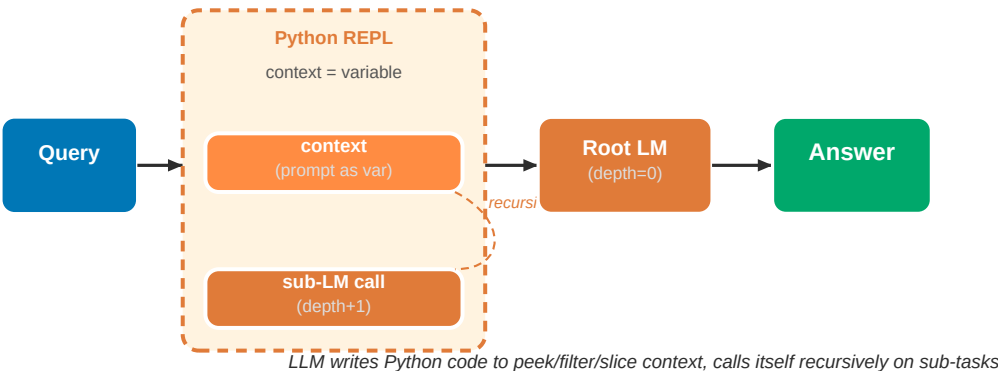
Paper: Zhang, Kraska & Khattab — *Recursive Language Models* (MIT CSAIL, Dec 2025)

arXiv: <https://arxiv.org/abs/2512.24601> | **GitHub:** <https://github.com/alexzhang13/rlm> | **Install:** `pip install rlm`

Core Concept

Instead of embedding and retrieving chunks (RAG), an RLM treats the entire input as a **Python variable in a REPL environment**. The LLM writes code to programmatically peek into, filter, slice, and grep through the context — then recursively calls sub-instances of itself on relevant fragments. No vector database, no chunking, no embedding model required.

Strategy 5: RLM Architecture



How It Works

- Query arrives. The RLM initialises a Python REPL where the full document is stored as a variable.
- The Root LM (depth=0) receives only the query + metadata (e.g. length). It never sees the raw context directly.
- The Root LM writes Python code: it can peek (slice), grep, or split the context string programmatically.
- It invokes recursive sub-LM calls (depth+1) on any substring — just like calling a function.
- Sub-calls return partial answers which the Root LM assembles into a final response.

Performance

Tested on GPT-5 and Qwen3-Coder-480B across contexts up to **10M+ tokens**. RLMs outperformed all baselines — including RAG agents, context compaction, and code-generation agents — by double-digit percentage gains on most tasks, at comparable or lower cost per query.

RLM vs RAG — Quick Comparison

Dimension	RAG	RLM
Retrieval mechanism	Vector similarity (embeddings)	Python code (grep / slice)
Preprocessing	Chunking + embedding index	None
Max context	Limited by index quality	Effectively unbounded
Context rot	Yes — retrieval errors compound	Mitigated by recursive decomp.
Cost model	Low per query (fast lookup)	Variable (API calls per sub-task)
LLM requirement	Any (even small models)	Strong coding ability needed
Best for	Known, well-structured corpora	Arbitrarily long, complex docs

Strengths & Weaknesses Comparison

Strategy	Strengths	Weaknesses	Best For
1. Router (Small Model)	• Very fast latency • Low compute cost • Simple to implement • Easy to debug	• Loses within-doc granularity • May need retraining • Binary doc selection	Clearly separable document topics
2. RAG + BM25 (Hybrid)	• Semantic + keyword match • Handles diverse queries • No preprocessing • Easy to update index	• Chunking artifacts • Flat hierarchy • May miss context • Reranker adds latency	Mixed query types, general retrieval
3. RAPTOR (Hierarchical)	• Multi-hop reasoning • Handles abstraction • Searches all levels • Good for synthesis	• High preprocessing cost • Summary drift risk • Complex to maintain • Slow index updates	Conceptual queries, cross-doc synthesis
4. ColPali (Visual Embed)	• Preserves layout/tables • No OCR errors • Works on scanned docs • Captures visual context	• Higher compute cost • Larger index size • Less mature tooling • Query needs VLM	Complex layouts, forms, invoices, scanned PDFs
5. RLM (Recursive LM)	• No chunking/embeddings • Handles 10M+ tokens • Adaptive decomposition • Context-rot resistant	• High API call cost • Variable latency • Needs capable LLM • Non-deterministic	Arbitrarily long docs, complex multi-step queries

Code Examples

ChromaDB — Simple Vector Store

```
import chromadb

client = chromadb.Client() # in-memory

collection = client.create_collection("docs")
collection.add(
    documents=["doc1", "doc2"],
    ids=["1", "2"]
)

results = collection.query(
    query_texts=["search term"],
    n_results=2
)
```

txtai — RAG with OpenRouter

```
import os
from txtai import Embeddings, LLM, RAG

os.environ["OPENROUTER_API_KEY"] = "your-key-here"

# LLM via OpenRouter (use litellm/ prefix)
llm = LLM("litellm/openrouter/meta-llama/llama-3.3-70b-instruct")

# Simple generation
response = llm("What is RAG?")
print(response)
```

RAPTOR — Hierarchical Retrieval with OpenRouter

```
import os
os.environ["OPENROUTER_API_KEY"] = "your-key"

from raptor import (
    BaseSummarizationModel, BaseQAModel, BaseEmbeddingModel,
    RetrievalAugmentationConfig, RetrievalAugmentation
)
from sentence_transformers import SentenceTransformer
from litellm import completion

# Embedding (local, free)
class LocalEmbedding(BaseEmbeddingModel):
    def __init__(self):
        self.model = SentenceTransformer("all-MiniLM-L6-v2")

    def create_embedding(self, text):
        return self.model.encode(text)

# Summarizer via OpenRouter
class OpenRouterSummarizer(BaseSummarizationModel):
    def summarize(self, context, max_tokens=150):
        resp = completion(
            model="openrouter/meta-llama/llama-3.3-70b-instruct",
            messages=[{"role": "user", "content": f"Summarize concisely:\n\n{context}"}],
            max_tokens=max_tokens
        )
        return resp.choices[0].message.content

# QA via OpenRouter
class OpenRouterQA(BaseQAModel):
    def answer_question(self, context, question):
        resp = completion(
            model="openrouter/meta-llama/llama-3.3-70b-instruct",
            messages=[{
                "role": "user",
                "content": f"Answer based on context only.\n\nContext: {context}\n\nQuestion: {question}"
            }]
        )
```

```

        }]
    )
    return resp.choices[0].message.content

# Setup
config = RetrievalAugmentationConfig(
    summarization_model=OpenRouterSummarizer(),
    qa_model=OpenRouterQA(),
    embedding_model=LocalEmbedding()
)
RA = RetrievalAugmentation(config=config)

# Index document
text = open("your_doc.txt").read()
RA.add_documents(text)

# Query
answer = RA.answer_question("What are the key points?")
print(answer)

```


Code Examples (continued)

RLM — Recursive Language Model (Strategy 5)

arXiv: arxiv.org/abs/2512.24601 | GitHub: github.com/alexzhang13/rlm | `pip install rlm`

```
# No chunking, no vector DB – the LLM navigates context with Python code from rlm
import RLM
rlm = RLM( backend="openai", backend_kwargs={"model_name": "gpt-4o"}, verbose=True, ) # Works on arbitrarily
long documents
huge_document = open("very_long_doc.txt").read() # millions of tokens: no problem
response = rlm.completion( f"Summarise the key arguments in this document:\n\n{huge_document}" )
print(response.response) # With OpenRouter (same API key as examples above):
rlm_or = RLM( backend="openai", backend_kwargs={ "model_name": "openai/gpt-4o", "api_base":
"https://openrouter.ai/api/v1", "api_key":
"k-or-v1-c2b3fed13934605c6b75e9962fbef4b867bf2da694d1661f5d88229cda0da3c", }, )
result = rlm_or.completion( "Find every mention of climate and summarise.\n\n" + long_text )
print(result.response)
```

References: Zhang, Kraska & Khattab (2025). *Recursive Language Models*. arXiv:2512.24601. MIT CSAIL. — github.com/alexzhang13/rlm